Introduction aux Bases de Données Relationnelles et Requêtes SQL de Base

Thème principal : Modèle relationnel, intégrité des données et langage SQL ANSI

Synthèse exécutive : Ce cours pose les fondations indispensables de la ressource R1.05 (Introduction aux Bases de Données) dispensée en BUT 1 Informatique à l'IUT de Bayonne et du Pays Basque. L'objectif pédagogique est de comprendre pourquoi nous abandonnons les tableurs plats (Excel) au profit des Systèmes de Gestion de Bases de Données Relationnelles (SGBD) afin d'éviter la redondance et les anomalies de mise à jour. Le cours détaille la structuration de l'information via les clés primaires et clés étrangères garantissant l'intégrité référentielle, et enseigne l'écriture de requêtes SQL d'extraction fiables et normalisées (norme ISO/ANSI) en utilisant les jointures explicites INNER JOIN.

Le Pourquoi : Des Tableurs aux Bases de Données Relationnelles

Dans les applications informatiques réelles, stocker des informations dans un fichier Excel ou un tableau plat pose rapidement des problèmes majeurs d'évolutivité et de fiabilité. Si une information (comme l'adresse d'un client) est répétée sur 50 lignes de commandes, la modifier nécessite 50 mises à jour distinctes. Si une seule ligne est oubliée, la base devient incohérente. Le modèle relationnel résout ce problème fondamental en séparant les entités distinctes (ex: 'Clients' d'un côté, 'Commandes' de l'autre) dans des tables dédiées et en établissant des liaisons logiques.

- Problématique des tableurs : Redondance de données et anomalies de mise à jour.
- SGBD (Système de Gestion de Bases de Données) : Logiciel garantissant la cohérence, la sécurité, l'accès concurrentiel et la persistance des données.
- Modèle Relationnel : Découpage des données en entités distinctes (tables) reliées logiquement pour éliminer les doublons.

Cas concret enseignant : *L'enseignant a illustré cela avec le système d'information bancaire : si un compte en banque stockait le nom du titulaire et son adresse sur chaque ligne de virement ou de paiement par carte, un simple déménagement exigerait de modifier des milliers d'historiques passés, générant un risque critique d'erreur de saisie et une saturation inutile du stockage.*

Clés Primaires, Clés Étrangères et Intégrité Référentielle

Chaque n-uplet (ligne) d'une table doit pouvoir être identifié de manière unique. La Clé Primaire remplit ce rôle : elle ne peut jamais contenir de valeur nulle (NOT NULL) ni être dupliquée. Pour lier deux entités (par exemple associer un étudiant à son groupe de TD), on n'insère pas toutes les informations du groupe dans la table Étudiant : on y stocke simplement l'identifiant du groupe sous forme de Clé Étrangère. Le SGBD applique alors automatiquement une contrainte d'intégrité référentielle.

- Clé Primaire (Primary Key - PK) : Identifiant unique de chaque ligne (n-uplet), obligatoire et non nul (NOT NULL).
- Clé Étrangère (Foreign Key - FK) : Attribut stocké dans une table qui fait référence à la clé primaire d'une autre table.
- Contrainte d'Intégrité Référentielle : Règle stricte gérée par le SGBD qui interdit d'insérer une clé étrangère pointant vers un identifiant inexistant.

Cas concret enseignant : *L'enseignant a cité l'analogie du Numéro de Sécurité Sociale (NIR) / INSEE à Bayonne : Même si deux étudiants portent exactement le même nom et le même prénom au Pays Basque, leur clé primaire unique empêche toute confusion ou doublon dans les systèmes administratifs.*

Extraction de Données en SQL ANSI : SELECT, WHERE et INNER JOIN

Le langage SQL (Structured Query Language) permet d'interroger la base de données. Pour croiser des informations entre deux tables, la norme moderne ISO/ANSI exige l'utilisation de la clause explicite INNER JOIN associée à la condition de liaison ON. La syntaxe standard s'écrit : SELECT colonnes FROM table1 INNER JOIN table2 ON table1.fk = table2.pk WHERE conditions. L'utilisation des alias de tables est fortement recommandée pour la lisibilité.

- Langage Déclaratif : Le SQL exprime CE QUE l'on cherche et non COMMENT l'extraire algorithmiquement.
- Projection (SELECT) : Spécification explicite des colonnes à extraire et à afficher.
- Filtrage (WHERE) : Application de critères logiques pour cibler uniquement les lignes souhaitées.
- Jointure Explicite (INNER JOIN ... ON) : Combinaison de deux tables basées sur la correspondance entre la clé étrangère et la clé primaire.

Cas concret enseignant : *Mise en garde stricte du professeur pour les examens : 'N'utilisez jamais la syntaxe obsolète du produit cartésien avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite INNER JOIN ... ON. Le produit cartésien implicite fait perdre des points en partiel et est banni en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on pas utiliser SELECT * dans un code applicatif ou lors d'un examen ?
Réponse du professeur : Parce que cela transfère des données inutiles sur le réseau, brise l'encapsulation si la structure de la table change (ajout/suppression de colonnes), et montre un manque de rigueur méthodologique sanctionné en partiel.

Question étudiant : Que se passe-t-il si j'essaie de supprimer un groupe dans lequel il y a encore des étudiants inscrits ?
Réponse du professeur : Le SGBD bloque immédiatement la suppression grâce à la contrainte d'intégrité référentielle, pour empêcher qu'un étudiant ne se retrouve rattaché à un groupe fantôme.

■ Exercice & Démonstration d'Application

Énoncé : On considère le schéma relationnel suivant : - Etudiant (id_etudiant, nom, prenom, #id_groupe) - Groupe (id_groupe, nom_groupe, promo) Écrire la requête SQL conforme à la norme ISO/ANSI permettant d'extraire le nom et le prénom des étudiants appartenant à la promotion 'BUT1-INFO' et inscrits dans le groupe nommé 'A1'.

Résolution pas à pas :
Étape 1 : Identifier les tables nécessaires et définir leurs alias -> Etudiant (e) et Groupe (g). Étape 2 : Définir la jointure explicite sur la correspondance clé étrangère / clé primaire -> INNER JOIN Groupe g ON e.id_groupe = g.id_groupe. Étape 3 : Appliquer les filtres dans la clause WHERE -> g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'. Étape 4 : Sélectionner uniquement les attributs demandés (projection) -> SELECT e.nom, e.prenom.

Résultat : SELECT e.nom, e.prenom FROM Etudiant e INNER JOIN Groupe g ON e.id_groupe = g.id_groupe WHERE g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1';

■ Auto-Évaluation Flash d'Examen

Q : Quelles sont les deux propriétés fondamentales d'une Clé Primaire (Primary Key) ?

R : Elle doit être unique (pas de doublons) et non nulle (NOT NULL).

Q : À quoi sert la clause INNER JOIN et sur quoi doit porter sa condition ON ?

R : Elle sert à combiner deux tables. Sa condition ON doit porter sur l'égalité entre la clé étrangère de la table fille et la clé primaire de la table parente.

Q : Pourquoi la syntaxe 'FROM TableA, TableB WHERE TableA.id = TableB.id_a' fait-elle perdre des points en examen ?

R : Il s'agit d'une syntaxe obsolète de produit cartésien implicite qui ne respecte pas les normes modernes ISO/ANSI exigées par l'équipe pédagogique et en entreprise.